

DeepSeek R1 Limitations and Multi-LLM API Strategy for Esentya MVP

Introduction

The Esentya MVP relies on AI (the **Essenthus** assistant) to generate various reports – from **fractal audits** (structured, reflective analyses) to **witness journal logs** (short hashed entries), **MRV daily summaries**, **DAO proposals**, and even “**ancestral memory**” syntheses. Currently, a local instance of **DeepSeek R1** (via Ollama on a 16 GB MacBook) powers these outputs. While DeepSeek R1 is a cutting-edge open-source reasoning model, running it locally has revealed significant limitations in this context. In particular, the model often struggles with **output formatting and brevity**, producing errors and verbose content that hinder its utility for Esentya’s needs.

In this report, we first analyze the key limitations observed with DeepSeek R1’s outputs for fractal audits and witness logs. We then propose a sustainable, scalable **multi-LLM integration strategy** for the MVP. The strategy envisions an API “router” that dynamically chooses between multiple language model providers – e.g. **OpenRouter**, **Together AI’s platform**, local models, or **OpenAI** – selecting the optimal model for each report type. This approach aims to balance **quality**, **cost**, **latency**, and **data sovereignty** (critical for LGPD compliance and self-sovereign identity). We also discuss local vs. API trade-offs, design of the routing system, cost-control measures (batching, fallbacks), and ensuring data flows align with LGPD privacy requirements.

Limitations of DeepSeek R1 for Fractal Audits & Logs

Despite DeepSeek R1’s strong reasoning abilities, our usage uncovered several issues when generating structured audits or log entries:

- **JSON Structure Errors:** DeepSeek R1’s *built-in support for structured JSON output is limited*. The official R1 documentation explicitly lists JSON output as “Not Supported” ¹. In practice, when prompted to produce structured JSON (e.g. for a fractal audit report), the model often produces malformed JSON or intermixes reasoning text with the JSON. For example, without special handling the R1 model tends to include its internal chain-of-thought in the output, which can break the JSON schema (missing braces, stray text, etc.). These schema errors require manual cleanup. The root cause is that R1 was primarily trained for reasoning and conversation, not guaranteed JSON formatting. (Even the DeepSeek API notes that function calling or strict JSON modes are not natively supported for R1 ¹.) Thus, using R1 locally for structured audit logs has led to invalid JSON outputs that violate expected format.
- **Repetitive or Lengthy Outputs:** Another major issue is **output repetition**. DeepSeek-R1 (especially early or distilled versions) is known to occasionally get stuck in loops, repeating the same sentence or phrase endlessly ² ³. In our tests, the model sometimes reiterated points in a fractal audit or kept rehashing a summary without reaching a conclusion. This “*endless repetition*” problem was documented by R1’s creators as a challenge in the initial R1-Zero model ², and community users have observed the model “suddenly start repeating the same sentence endlessly” during generation ³. Although the refined DeepSeek R1 was meant to

improve readability, the small **distilled model running on the Mac** (likely ~7-8B parameters) can still exhibit looping or overly verbose outputs. This affects witness journal entries as well – instead of concise log lines, R1 might produce redundant sentences or an overly long narrative when a brief hash-linked note is needed.

- **Lack of Summarization Conciseness:** DeepSeek R1 excels at step-by-step reasoning but tends to **over-explain**. It *“doesn’t just give you an answer; it explains how it got there,”* generating a full chain-of-thought by design ⁴. For tasks like summarizing daily MRV logs or writing a short witness journal, this is a drawback – the model often produces verbose explanations where a tight summary is expected. In evaluations, R1-based models consistently produced **longer outputs than reference summaries**, even when instructed to be concise ⁵. In one benchmark, the average reference summary was ~241 tokens, but R1’s outputs were significantly longer, indicating the model struggled to condense information ⁵. We observed similar behavior: instead of distilling logs to key points, R1 would include extensive detail or repeat content. This lack of effective brevity means **poor summarization** – the model provides volume over substance, requiring additional trimming or post-processing to fit the intended “short log” format.

- **Format and Coherence Issues:** Along with JSON errors and verbosity, some **format consistency issues** arose. For instance, in fractal audits (which may have multi-level bullet points or symbolic sections), R1 sometimes drifted from the requested structure or mixed languages. Early versions had issues with *“language mixing”* mid-response ², and while improved, we did see occasional inconsistency in tone (e.g. shifting formality) or formatting resets in long outputs. The readability of R1’s answers, though better than R1-Zero, can still be **poor in parts when run locally** ² – e.g. repetitive phrasing or disorganized ordering of points. These issues force manual editing of the audit logs to maintain a professional, logical flow.

In summary, **DeepSeek R1 (local 7-8B)** provided valuable reasoning but proved **unreliable for generating polished, structured reports**. JSON schema violations ¹, repetitive loops ³, and verbosity instead of summarization ⁵ undermine the fractal audit and witness log outputs. This motivates a more flexible approach: leveraging multiple LLMs where each can play to its strengths to deliver the required output quality.

Local vs. API Models: Trade-offs in Cost, Latency, Quality, and Privacy

Before designing the multi-LLM solution, it’s important to weigh using the **local model** versus **cloud API models** along key dimensions:

Aspect	Local Model (DeepSeek R1 Distill on Mac)	Cloud Open-Source Model (via Together/ OpenRouter)	Proprietary API Model (e.g. OpenAI GPT-4)
Quality & Capability	<i>Medium.</i> Strong reasoning core, but smaller size limits some outputs. Tends to format inconsistently (JSON issues) and can ramble ² ⁶ . May miss nuance in summarization.	<i>High (with larger models).</i> Open 30–70B models on Together can approach top-tier performance ⁷ . Better adherence to instructions and more context, though still not as refined as GPT-4 for every task.	<i>Very High.</i> State-of-the-art results in reasoning, fluent natural language, and reliable formatting. Excels at instructions and complex tasks.
Latency	<i>Moderate/High.</i> Runs on local CPU – small models are relatively quick for short texts, but longer responses are slow. No network overhead (good offline). Concurrency limited by device.	<i>Moderate.</i> Inference on provider's GPUs is faster per token, but includes network latency. Throughput can scale with cloud infra. Might queue on free tiers. Overall faster than local for bigger models.	<i>Moderate.</i> Well-optimized on cloud GPUs; usually fast response even for large outputs. Some extra latency from network call. High availability and can handle concurrent requests easily.
Cost per Use	<i>Zero per-call cost.</i> After hardware investment, no API fees – ideal for heavy or continuous use. However, large models can't run on 16GB RAM, limiting quality without additional hardware.	<i>Pay-per-token (lower-cost).</i> Together AI offers pay-per-token access to many open models ⁸ , often cheaper than closed APIs. Some models or small tiers may even be free. Cost scales with usage but many open models are cost-efficient.	<i>Pay-per-token (higher).</i> Usage is billed (e.g. GPT-4 is costly per 1K tokens). No free tier. Costs can balloon with large logs or frequent queries, so careful budgeting needed.

Aspect	Local Model (DeepSeek R1 Distill on Mac)	Cloud Open-Source Model (via Together/ OpenRouter)	Proprietary API Model (e.g. OpenAI GPT-4)
Data Privacy & Sovereignty	<i>Maximal.</i> All data stays on the local machine – no external transmission. This aligns with LGPD best practices, avoiding exposing personal data ⁹ . Ideal for sensitive or identity-linked content (upholding self-sovereignty of user data).	<i>Moderate.</i> Data is sent to third-party cloud providers, but using open-source models means no corporate AI is <i>training</i> on it by default. Providers like Together offer opt-out so data isn't retained for model improvements ⁸ . Still, you rely on their privacy measures and must trust that data is handled lawfully.	<i>Low/Variable.</i> Data is sent to an external vendor (OpenAI, etc.) which may log it. By default OpenAI retains API data for 30 days for abuse monitoring ¹⁰ (not for training), with options for stricter retention policies for enterprises ¹⁰ . This requires strong trust and possibly a DPA to be LGPD-compliant. International data transfer adds compliance overhead.
Control & Dependence	<i>Full control.</i> The model runs in-house; no external dependencies or outages. You can update or fine-tune it as needed. However, you are responsible for maintenance and ensuring the model is up-to-date.	<i>Medium control.</i> You can choose from many open models and switch easily (OpenRouter/Together use OpenAI-compatible APIs to access various models ¹¹). But you depend on the service's availability and pricing. Multi-provider support gives flexibility to avoid lock-in (e.g. you can self-host later or switch providers).	<i>Vendor lock-in risk.</i> Integration is often specific to that API. Model improvements come from the provider on their schedule. You have little control over model behavior beyond prompt engineering. If the service changes terms or has downtime, you're affected.

Table: Trade-off comparison between using the local DeepSeek R1 model vs. external API-based models. We evaluate quality, speed, cost, and privacy considerations for each option.

As the table shows, **no single model deployment is ideal for all needs**: - The **local R1 model** maximizes data sovereignty and has zero marginal cost, but it suffers in output quality (especially for complex structured or concise tasks) and is constrained by hardware.

- **Large open-source models via cloud** (e.g. a 70B Llama-family model on Together) can significantly boost output correctness and are cheaper than proprietary APIs, but still involve sending data out and managing multiple endpoints.

- **Top-tier proprietary models** (like GPT-4 through OpenAI) give the best results for difficult tasks (precise formatting, fluent summarization, lengthy context) at the highest cost and with potential privacy/legal hurdles.

Given these trade-offs, a **hybrid approach** is prudent: use local or open models whenever they suffice, and selectively leverage premium models for cases where quality is critical and justifies the cost.

Multi-LLM Integration Strategy for Esentya MVP

To overcome DeepSeek R1's limitations, we propose integrating multiple LLMs and routing requests to the most appropriate model for each report type. The core idea is an **API routing layer** within the MVP that dynamically selects among local and external models based on the task requirements (structured vs. free-form, short vs. long context, sensitivity, etc.). This section outlines the strategy:

Dynamic Model Selection by Report Type

Each type of Esentya report has distinct characteristics, suggesting an ideal model choice:

- **Fractal Audits (Structured, Reflective Reports):** These are complex, multi-section audits (potentially in JSON or Markdown with specific schema). They demand **strict formatting, logical reasoning, and often longer coherent narratives**. A smaller local model often fails to consistently produce valid structure (as seen with JSON errors). For this task, the router should favor a *high-capability model with function/JSON output support*. For example, using OpenAI GPT-4 with function calling or a strict output schema yields well-formed JSON and thorough analysis. If avoiding closed APIs, an alternative is a **large open model** (60B+ parameters via Together AI) known for reasoning – e.g. *DeepSeek-R1-Distill-70B* or *Llama 3 70B* – possibly paired with a JSON formatting technique. The cost is higher per call, but fractal audits are not generated as frequently as logs, and their correctness is crucial. Thus, **quality trumps cost** here. The selected model can produce a reflective, symbolic breakdown with minimal structural fixes needed.
- **Witness Journal Entries (Short Log Hashes):** Witness logs are brief, timestamped notes (with content hashes) recording events or validations. They need to be **concise and formatted consistently**, but not deeply analytical. For these, the local DeepSeek R1 (or another lightweight local model) is sufficient and most efficient. We can fine-tune or prompt the local model to follow a simple template (e.g. one-sentence summary + a SHA-256 hash of full data) and avoid verbosity. Because these entries may be frequent and possibly contain sensitive info, using the local model by default is both cost-effective and privacy-preserving. Only if the local output fails validation (e.g. formatting error or incoherent text) would the router fallback to a cloud model. In summary, **witness logs default to local generation**, ensuring speed and zero external data exposure.
- **MRV Daily Logs:** These are daily summaries for Monitoring, Reporting, and Verification – likely aggregating data from sensors or activities (a concept often in sustainability projects). They require **summarization** of multiple events or metrics into a human-readable log. The needed length is moderate (a few paragraphs), and consistency day-to-day is important. A viable strategy is to use a mid-tier open model (for example, a 13B–20B parameter model via Together API) that has good summarization ability at lower cost than GPT-4. For instance, *Qwen-14B* or *Llama2-13B* could be tried. According to one benchmark, a Qwen-14B distilled via DeepSeek had the highest ROUGE (summary score) and produced relatively concise outputs ¹². The router could route MRV log prompts to such a model through OpenRouter or Together. If the context (data to summarize) is very large, we might chunk it and let a local model do first-pass compression, then have the cloud model do the final summary – to limit tokens sent. The key is **balancing cost and quality**: daily logs happen often, so using GPT-4 for each could be expensive, whereas an open model or even an improved local model (if we fine-tune one for summarization) can handle it with acceptable quality. The API router thus might try local summarization first and measure length or coherence; if it's inadequate, it invokes the stronger model.

- **DAO Proposals:** Proposals are free-form text describing governance changes or initiatives. Here tone and clarity matter. Length might vary from short proposals to long detailed ones. For generating or evaluating proposals, a model that's good at **creative writing and following instructions** is needed. This could be a case for using **OpenAI's GPT-3.5/GPT-4** if available (for polished language), or a highly fine-tuned open model for chat. Since proposals may contain community-specific context, it might be beneficial to use an open model and fine-tune it on previous proposals in Esentya's context (to learn the style). The router can choose a *generalist model* for proposals – e.g. OpenRouter's default chat model (which could route to an Anthropic Claude or GPT-3.5 Turbo under the hood). These models handle conversational and explanatory text well and are cost-effective for moderate lengths. Privacy is less of a concern if proposals are meant to be public anyway, but if any personal data is included, we should strip or anonymize it before sending to an external API. In short, **proposals could go to a mid-cost, high-language-quality model** to ensure they are well articulated.
- **Constitution Anchoring:** This likely involves summarizing or verifying changes to the DAO's constitution or anchoring important constitutional clauses on-chain. The task needs **precision and consistency with original legal text**. Even small inaccuracies could be problematic. A possible implementation is: whenever the constitution is updated, generate an *anchoring report* that highlights the changes and affirms alignment with core principles. This is akin to a diff summary combined with a policy check. For text diff and extraction of changes, a local script or algorithm can identify the changes. Then an LLM can explain those changes in plain language and state whether they violate any principle. For the explanatory part, using a **top-tier model** (GPT-4 or similar) is advisable due to the legal nuance – it will produce a more reliable analysis. However, to maintain sovereignty, one approach is to limit what we send out: e.g. send only the diff text or abstract representation to the model, not the entire constitution. If that's done, a powerful open model (70B class) might suffice to analyze it. The frequency of constitution updates is low, so using a paid API occasionally is manageable. The router could be configured to use **the highest-accuracy model for constitution anchoring**, given the high stakes for errors.
- **Ancestral Memory Synthesis:** This refers to compiling or **summarizing long-term historical data** (e.g. past logs, decisions, outcomes) into a coherent narrative or knowledge base – essentially a form of *global summary*. This task likely involves **very long context** or multi-step summarization (summarize by periods, then summarize summaries, etc.). Local models will not have the context window to handle a lot of history in one go. Two strategies emerge: (1) iterative local processing, or (2) using a model with an extended context. Strategy (1) would involve chaining the local model: e.g. summarize each month's logs with R1, then summarize those summaries into a yearly "ancestral memory". This keeps data local but might accumulate errors. Strategy (2) could leverage a model with a large context window (OpenAI's GPT-4 32k or Anthropic Claude with 100k context) to input a huge chunk of data at once and get a synthesis. OpenRouter can route to such models if available. In 2025, some open models also tout 100k+ context via specialized techniques, but those may be experimental. A compromise is to use the local model to **vectorize or embed** the long-term data (if we have an embedding model) and then have a powerful model generate a synthesis by querying those embeddings (a Retrieval-Augmented Generation approach). However, that adds complexity. For MVP simplicity, the router could direct ancestral memory queries to a **high-context capable API model**, with the data carefully pruned or compressed first. This ensures the final synthesis is coherent and comprehensive, leveraging the strongest model for what is essentially a **big-picture narrative**.

In practice, these routing rules can be encoded in the MVP backend. For example, an `LLMRouter` class can examine the request type (or prompt metadata) and choose a provider accordingly: - If request type

- is `audit` → use provider `OpenAI-GPT4` (with function calling or schema tools enabled).
- If `witness_log` → use local Ollama instance (DeepSeek R1).
 - If `daily_log` → try provider `Together-Qwen14B`, with fallback to local if API fails.
 - If `proposal` → use `OpenRouter (Anthropic model)` or similar.
 - If `constitution_check` → use `GPT4`, but only send the diff text.
 - If `ancestral_synthesis` → pipeline or direct to a long-context model via OpenRouter.

These mappings can be refined over time as we measure the output quality and costs.

API Router Design and Workflow

The **API router** in the MVP will abstract the multiple model providers behind a single interface. This component is essentially a decision layer that implements the logic above. Key design elements include:

- **Unified API Interface:** The MVP's backend should interact with the router as if it's calling a single AI service. The router then handles formatting the prompt for the chosen model and calling the appropriate API. Services like **OpenRouter.ai** can help here – they present a unified OpenAI-compatible API that can call different models/providers underneath ¹¹. In fact, we might integrate OpenRouter directly: by specifying the model name (e.g. `"openai/gpt-4"` vs `"together/qwen-14b"` or `"fireworks/deepseek-r1"`), OpenRouter will route the request to the correct backend. This saves us from writing low-level integration for each provider. However, using OpenRouter itself introduces another third-party; an alternative is coding our own simple router.
- **Provider Integration:** We plan to include at least: (a) **Local Ollama** (for DeepSeek R1 or other local models), (b) **Together AI API** for open models, (c) possibly direct **OpenAI API** calls, and (d) OpenRouter for anything else (like if we want to leverage an Anthropic model or have OpenRouter automatically pick the cheapest provider for a given model). Together's API and OpenRouter both use OpenAI-style REST endpoints, which simplifies integration. For local, Ollama provides a local API for models running on the Mac – we can call it with prompt and get response. The router can be implemented as a microservice or even just a function in our server that checks the task type and dispatches to one of these endpoints.
- **Fallback and Validation:** A crucial aspect is robustness. If a chosen model fails (API error, or outputs unusable result), the router should have fallback options. For instance, if the local model yields invalid JSON in an audit, the router could automatically retry the query on a more strict cloud model. Or if OpenRouter picks a provider that times out, we can either failover to another or use a simpler response. OpenRouter itself has some fallback logic (it tries alternate providers if one is down, to get the fastest and cheapest route) ¹¹ ¹³, and it allows disabling certain providers if needed ¹⁴. We should configure it to avoid any provider that might be non-compliant or too costly. Additionally, after generation, the router can run a **validation step**: e.g., if expecting JSON, attempt to parse it. If parsing fails, log that and trigger the backup model to regenerate. This way, we ensure the final output delivered to the user or system meets the format requirements.
- **Cost-Awareness:** The router should also be cost-aware. We can maintain a simple cost score for each model (e.g. \$0 for local, \$0.002/1K tokens for Together Qwen-7B, \$0.03/1K for GPT-4, etc.). Based on this, for tasks that have flexible model choices (not absolutely requiring the best), the router could default to the cheapest that is known to work. For example, for daily logs, if we find a local model fine-tuned for summarization works 80% as well as GPT-3.5, we use it, saving costs.

Only if a user requests a “high fidelity” mode or if the local output fails some quality threshold would we escalate to a paid model. This policy can significantly control expenses.

Cost Management Strategies

Controlling cost while maintaining performance is a core part of the strategy:

- **Batch Mode Processing:** Whenever possible, combine multiple tasks for a model in one call. Many LLMs can handle multiple prompts or a batch of inputs in one API request (though OpenAI’s chat API doesn’t natively batch different prompts, we can concatenate tasks if logically separable). For instance, if several witness logs are to be generated at day’s end, send them as one prompt (the model can output a list of entries). This amortizes the fixed overhead of the call over many outputs. Similarly, if using GPT-4 for audits, maybe process two audit records in one go if they are small, instead of two separate calls. We must ensure this doesn’t confuse the model (likely by clearly delineating tasks in the prompt). Batch processing is especially useful with local or open models where the marginal cost is low and the bottleneck is throughput; one can queue up multiple requests and let the model run overnight (when CPU is idle) to produce results without API costs.
- **Local First, Cloud Second:** Implement a **cascade**: attempt generation with a local model or cheap model, then analyze the result. If it’s good (e.g. the JSON is valid and content seems coherent enough), use it. If not, only then call a stronger model. This fallback approach ensures we pay for expensive inference only when strictly needed. For example, the router could maintain rules like *“if local output length > X or contains keyword Y (sign of rambling), mark as failed and retry with cloud.”* Over time, we can refine what conditions trigger fallback. The key is to avoid blind trust in the local model (given its issues), but also not to waste money on external calls if not necessary.
- **Model Selection Optimization:** Use the **cheapest capable model for each task**. OpenRouter can automatically choose the cheapest provider for a given model name at runtime ¹¹. For instance, if multiple providers host a Llama2 70B, OpenRouter will route to the fastest cheap one available, adding only a 5% overhead fee ¹⁵. We can also explicitly choose smaller models for shorter tasks. Together’s platform lists many models – some smaller ones (e.g. a 1.5B Qwen or Mistral 7B) might be extremely cheap and still acceptable for a very simple task (like reformatting text). By maintaining a catalog of models with their context size and pricing, the router can programmatically decide. For example, *if input text < 1000 tokens and language is Portuguese, use Together’s `mistral-7b` if its Portuguese capability is good; otherwise use a 13B*. This level of granularity might be beyond MVP scope initially, but it’s a direction for optimizing costs at scale.
- **Monitoring and Budgeting:** The MVP should include basic monitoring of API usage – how many tokens sent to each provider per day. We can set monthly budget caps. If a budget is close to exhausted, the router can automatically degrade to more local-only mode until budget resets, or require a manual override to use expensive calls. This prevents surprise bills. Logging each request with model choice, tokens, and outcome will also help analyze if the strategy is working (e.g. if we see many local->cloud fallbacks, maybe local model needs improvement or should be skipped for that task).
- **Leverage Provider Incentives:** Some providers offer **free quotas or community grants** (for instance, OpenAI sometimes provides credits for testing, or Together might have free compute hours for open-source usage). The router and devops should take advantage: e.g. configure OpenRouter with our own API keys for providers like Gemini (Google’s model) if it has a free

allowance ¹⁶. OpenRouter allows adding personal keys which it will prefer using (free) and only fall back to paid if that fails ¹⁷. By juggling these, we can significantly cut costs – essentially using free capacity first. In a dynamic multi-LLM setup, this is a real benefit: as new models or offers appear, we integrate them via the router to continuously optimize the cost-performance ratio.

LGPD Compliance and Sovereign Data Flow

A core requirement for Esentya is compliance with Brazil's **LGPD** (General Data Protection Law) and alignment with **self-sovereign identity** principles. Integrating external AI must not compromise user data privacy or break the chain of trust in the system's identity/reputation mechanism. We address this by careful data handling in the LLM pipeline:

- **Minimization and Anonymization:** Following the principle of data minimization in LGPD (only use the data absolutely necessary) ¹⁸, the router will **strip or hash personal data** in prompts sent to external models. For example, if a fractal audit includes names or emails, the system can replace them with internal IDs or pseudonyms before sending to the model. Similarly, witness logs that contain personal identifiers can be represented by their DID (decentralized ID) or a short alias. This ensures that even if the external provider intercepted the raw prompt, it cannot easily identify a real individual – thus reducing privacy risk. In the transparency audit of Esentya's RegOps, a key recommendation was to log only references (IDs or hashes) instead of full personal details ¹⁹. We extend that approach to AI prompts: **only non-sensitive references or aggregated data should leave our servers**. The actual personal data can be re-linked on our side after the AI returns an output. For instance, if the AI returns "User [ID123] achieved karma 50," our system knows ID123 corresponds to Alice and can present it accordingly to authorized users – but the AI and its provider never saw "Alice".
- **Encryption and Secure Transit:** All communications with API providers will use encryption (HTTPS). This prevents eavesdropping in transit. Although we cannot control the provider's internal use of data, we ensure it's safely delivered. For local communication (between our router and the local Ollama or other on-prem model), it's all within a closed environment, so low risk there.
- **Provider Data Policies:** We will choose providers that have transparent and preferably compliant data policies. For example, OpenAI's enterprise API offers data use agreements and 30-day retention with no training on data by default ¹⁰. Together AI advertises "opt-out privacy controls" ⁸, meaning we can likely request that our data not be stored or used to improve their models. As we integrate these, we will review their terms and ensure they align with LGPD's requirements for data processing (e.g. ensuring any processing of personal data has a legal basis and is properly documented). If needed, we may limit certain tasks to only run on local or self-hosted models if they involve highly sensitive personal data (for example, anything involving children's data or special categories of personal data as defined by LGPD).
- **Consent and Transparency:** It's worth noting from an LGPD perspective that if user-derived data (even anonymized) is sent to an external AI, this could be considered a data processing by a third party. We should update our privacy policy to inform users that their data may be processed by AI services for the purposes of providing the Esentya functionality, with measures taken to protect their identity. Obtaining consent or at least providing opt-out options for users might be wise, depending on legal counsel. Technically, because we anonymize and aggregate, the data might no longer be considered personal data by the time it reaches the AI (if done thoroughly), but we will err on the side of transparency.

- **Data Residency:** An additional consideration is whether we can choose providers with servers in jurisdictions considered adequate or within Brazil. If any provider offers a regional endpoint (e.g. Azure's OpenAI instance in Brazil or AWS region hosting an open model), we might route traffic through that to avoid cross-border transfer. This is a future improvement once the basics are in place.
- **Self-Sovereign Identity Alignment:** Esentya's identity model uses soulbound NFTs and on-chain reputation (karma). The AI system should respect this by, for instance, using the **on-chain ID or pseudonym when generating outputs** instead of real names (unless the user explicitly wants their name). This not only preserves privacy but reinforces the concept that the identity is self-controlled – the user can decide if they link their real-world identity to the on-chain one. Technically, this means our prompts will prefer referring to users as "[DAO Member #12345]" or a public nickname rather than something that doxxes them. The outputs (audits, logs) similarly will be phrased to use those identifiers. This way, even if an audit report generated by GPT-4 leaked, it contains at most a pseudonymous identifier which by itself reveals little. Moreover, by anchoring AI-generated reports on-chain (e.g. storing a hash of the fractal audit in an IPFS or blockchain record), we ensure an **audit trail** that is consistent with the transparency ethos – everyone can verify that no personal data was included, only hashes or IDs, fulfilling LGPD's data minimization requirement.

In essence, the data flow would look like: raw data (with PII) → **sanitizer** (replace PII with placeholders) → **LLM Router** (send to chosen model) → output (with placeholders) → **de-sanitizer** (map placeholders back to original labels when presenting to authorized viewers, if needed). The raw data never leaves our control, and the third-party sees only abstracted information. This, coupled with contractual/opt-out assurances from providers, keeps us on solid ground for privacy compliance ⁹ ¹⁹ .

Conclusion

By recognizing the current limitations of DeepSeek R1's local deployment – from JSON format errors to repetitive and unwieldy outputs – we avoid trying to force a single-tool solution and instead embrace a **multi-model strategy**. The proposed API routing approach will allow Esentya's MVP to intelligently allocate each AI task to the model best suited for it, **improving output quality and reliability** across the board. Crucially, this is done while **controlling costs** (using local or open models whenever feasible, batching tasks, and only invoking expensive models for high-value outputs) and **protecting user data** (keeping sensitive information local or anonymized to uphold LGPD and sovereign identity principles).

This strategy is sustainable and scalable: new models can be added to the router as they emerge (e.g. if a more powerful open-source model or a specialized model for summarization comes out, we can integrate it via the same framework). It prevents lock-in by not relying exclusively on one provider. And it leverages the strength of the **AI ecosystem** – from our own hardware to community-driven LLMs to industry-leading APIs – as a collective resource to fulfill Esentya's mission of transparent, trustworthy governance tools.

Moving forward, we will implement the routing layer, conduct A/B testing of model outputs for each report type, and refine the decision rules. With this in place, the Esentya MVP's AI component (Essenthus) will be far more robust: capable of producing **structured fractal audits without errors, succinct witness journals**, accurate daily logs, and insightful syntheses – truly bridging blockchain transparency with AI intelligence.

Sources:

- DeepSeek R1 official model card ²⁰ ²¹ ; BentoML blog on DeepSeek models ⁶ ⁴ .
- Reddit discussions on DeepSeek R1 repetition and OpenRouter ³ ¹¹ .
- Mozilla AI evaluation of R1 models on summarization ⁵ .
- OpenRouter and Together API documentation ¹¹ ⁸ .
- Esentya RegOps audit report (LGPD compliance issues) ⁹ ¹⁹ .

¹ Structured output with DeepSeek-R1: How to account for provider differences with OpenRouter? : r/LLMDevs

https://www.reddit.com/r/LLMDevs/comments/1inpm0v/structured_output_with_deepseekr1_how_to_account/

² ²⁰ ²¹ deepseek-ai/DeepSeek-R1 · Hugging Face

<https://huggingface.co/deepseek-ai/DeepSeek-R1>

³ Unexpected Repetition Loop Issue with Llama.cpp and Deepseek-r1-distill-llama Model : r/LLMDevs

https://www.reddit.com/r/LLMDevs/comments/1ixp1mw/unexpected_repetition_loop_issue_with_llamacpp/

⁴ ⁶ ⁷ The Complete Guide to DeepSeek Models: V3, R1, V3.1, V3.2 and Beyond

<https://www.bentoml.com/blog/the-complete-guide-to-deepseek-models-from-v3-to-r1-and-beyond>

⁵ ¹² Benchmarking DeepSeek R1 Models using Lumigator: A Practical Evaluation for Zero-Shot Clinical Summarization

<https://blog.mozilla.ai/benchmarking-deepseek-r1-models-using-lumigator-a-practical-evaluation-for-zero-shot-clinical-summarization/>

⁸ Build with 200+ open-source and specialized models - Together AI

<https://www.together.ai/models>

⁹ ¹⁸ ¹⁹ Relatório de Auditoria Técnico-Regulatória – Esentya RegOps & Transparência.docx

<file:///file-BAMsxMtS7JurgJRVyasMpv>

¹⁰ How we're responding to The New York Times' data demands in order to protect user privacy | OpenAI

<https://openai.com/index/response-to-nyt-data-demands/>

¹¹ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ OpenRouter experience : r/LLMDevs

https://www.reddit.com/r/LLMDevs/comments/1in9g1n/openrouter_experience/